

1. Title Page

Course Name: Object-Oriented Programming (Java)

Project Title: Smart Expense Tracker

Student Name: Yaman Yusuf

Roll Number: _77_____

Instructor Name: Mr_Susovan Jana_____

Semester: Spring 2026

2. Problem Statement (Max 1 Page)

In today's fast-paced world, managing personal finances has become increasingly complex. Many individuals fail to keep track of their income and expenses, leading to poor budgeting and financial instability. Traditional methods such as manual record-keeping are inefficient and prone to errors.

The objective of this project is to design and implement a **Smart Expense Tracker** using Java, based on Object-Oriented Programming principles. The system enables users to record, categorize, and analyze their financial transactions efficiently.

The application provides functionalities such as:

- Adding income and expense transactions
- Viewing transaction history
- Calculating total income, expenses, and balance
- Searching transactions by category

This system aims to offer a structured and scalable solution for managing personal finances.

3. System Design

3.1 UML Class Diagram (Text Representation)

Transaction (abstract)

amount

category

date

getters

display()

↑

|

|

Income

Expense

(display +)

(display -)

ExpenseManager

ArrayList<Transaction>

addTransaction()

displayAll()

getTotalIncome()

getTotalExpense()

getBalance()

searchByCategory()

Main

main()

3.2 Description of Major Classes

◆ Transaction Class

- Abstract base class
- Contains common attributes: amount, category, date
- Defines abstract method display()

◆ Income Class

- Inherits Transaction
- Represents incoming money
- Overrides display() to show positive amount

◆ Expense Class

- Inherits Transaction
- Represents outgoing money
- Overrides display() to show negative amount

◆ ExpenseManager Class

- Core logic handler

- Stores all transactions
 - Performs calculations and search operations
-

◆ Main Class

- Entry point of program
 - Handles user interaction
 - Controls application flow
-

3.3 Package Structure

project/

├— Transaction.java

├— Income.java

├— Expense.java

├— ExpenseManager.java

└— Main.java

3.4 Interaction Flow

User Input



Main.java



Create Object (Income/Expense)



ExpenseManager



Processing (store / calculate / search)



Output Display

4. OOP Concepts Demonstrated

4.1 Abstraction

Where used: Transaction class

Why:

To define a common structure while hiding implementation details

Snippet:

```
public abstract class Transaction {  
    public abstract void display();  
}
```

Explanation:

The abstract method forces child classes to define their own display logic.

4.2 Inheritance

Where used: Income and Expense

Why:

To reuse code and avoid duplication

Snippet:

```
public class Expense extends Transaction {  
    public Expense(double amount, String category, String date) {  
        super(amount, category, date);  
    }  
}
```

```
}
```

Explanation:

Child classes inherit properties and behavior from Transaction.

4.3 Polymorphism

Where used: displayAll()

Why:

To allow same method call to behave differently

Snippet:

```
for (Transaction t : transactions) {  
    t.display();  
}
```

Explanation:

Runtime decides which display() to execute.

4.4 Encapsulation

Where used: Transaction

Why:

To protect data

Snippet:

```
private double amount;  
  
public double getAmount() {  
    return amount;  
}
```

Explanation:

Direct access is restricted; controlled access provided.

5. Implementation Highlights

5.1 Use of ArrayList

`ArrayList<Transaction> transactions;`

👉 Allows dynamic storage of multiple transaction types

5.2 Runtime Type Checking

`if (t instanceof Income)`

👉 Separates income from expense during calculations

5.3 Input Validation

`if (amount <= 0)`

👉 Prevents invalid entries

5.4 Exception Handling

`catch (InputMismatchException e)`

👉 Prevents program crash

6. Testing & Error Handling

6.1 Test Cases

Case	Input	Output
Add Income	5000	Stored
Add Expense	2000	Stored
Summary	-	Correct result

6.2 Edge Cases

- Negative amount ❌
 - Empty category ❌
 - No transactions ✓ message
-

6.3 Failure Handling

- Invalid input handled
 - Safe program execution
-

7. Git Workflow

Development Steps

1. Project initialization
 2. Transaction class creation
 3. Income & Expense implementation
 4. ExpenseManager logic
 5. Main UI development
 6. Testing and bug fixes
-

Version Control Benefits

- Track changes
 - Maintain history
 - Enable collaboration
-

8. Conclusion

The Smart Expense Tracker demonstrates effective use of Object-Oriented Programming concepts. The system is modular, scalable, and easy to extend. It successfully handles financial data in a structured manner.

9. Future Scope

- GUI using JavaFX
 - Database integration
 - Graphical reports
 - Mobile app version
 - Authentication system
-

Appendix

A. Project Proposal

(Attach your proposal)

B. Approval Mail

(Attach screenshot)

C. Additional Submissions

(If applicable)

 FINAL NOTE

 Ye content **actual 10–12 pages cover karega** jab:

- Font: Times New Roman
 - Size: 12
 - Line spacing: 1.5
-